

Introduction à la Programmation MPI
Contrôle continu du 24/11/2023 – Ce document contient 4 pages
1h30 – Tout document autorisé (sauf la copie du voisin)

Exercice I : Petit programme MPI

On considère le programme MPI suivant :

```
1. #include <mpi.h>
2.
3. int var_glob = 0;
4.
5. int main(int argc, char **argv) {
6.     int rang, P;
7.
8.     MPI_Init(&argc, &argv);
9.
10.    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
11.    MPI_Comm_size(MPI_COMM_WORLD, &P);
12.
13.    if (rang == 0)
14.        var_glob = 1;
15.
16.    MPI_Barrier(MPI_COMM_WORLD);
17.
18.    var_glob = var_glob + 1;
19.
20.    if (rang == 1)
21.        printf("var_glob=%d\n", var_glob);
22.
23.    MPI_Finalize();
24.
25.    return 0;
26. }
```

Questions :

- a) Suite à l'exécution de ces lignes, quelle est la valeur de la variable `var_glob` en fonction du rang du processus ? Justifiez brièvement votre réponse.
- b) Modifiez le programme pour que le processus de rang 1 affiche la valeur affectée par le processus de rang 0.

Exercice II : Correct/Pas correct

Pour chacune des sections de code suivantes (numérotées de **a** à **e**), dites :

- si elle fait l'objet d'un risque de blocage (« deadlock ») ;
- ou bien si les contenus des messages risquent d'être erronés (« erroné ») ;
- ou bien si la section est sûre (« correct »).

Justifiez vos choix.

Dans le cas où une section de code n'est pas correcte, apportez une correction pour que la section soit sûre.

Dans tout ce qui suit, on désigne par rang le rang du processus MPI, et par P la taille du communicateur MPI_COMM_WORLD. On suppose que le programme a correctement initialisé MPI.

a) But : faire circuler un message (le contenu de m) dans un anneau initialisé par le processus 0. Le message circule dans le sens croissant des rangs modulo P.

```
int droite, gauche, m;
if (rang == 0) {
    m = 0;
}
droite = (rang + 1) % P;
gauche = (rang + P - 1) % P;
MPI_Recv(&m, 1, MPI_INT, gauche, 1000, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
MPI_Send(&m, 1, MPI_INT, droite, 1000, MPI_COMM_WORLD);
```

b) Le processus 0 veut distribuer un entier différent à chacun des autres processus

```
MPI_Request req[P];
int entier;
if (rang == 0) {
    for (i = 1; i < P; i++) {
        entier = i*i;
        MPI_Isend(&entier, 1, MPI_INT, i, 1000, MPI_COMM_WORLD, req+(i-1));
    }
    MPI_Waitall(P-1, req, MPI_STATUSES_IGNORE);
} else {
    MPI_Recv(&entier, 1, MPI_INT, 0, 1000, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}
```

c) Le processus 0 diffuse une valeur aux autres processus.

```
int som = 1;
if (rang == 0)
    MPI_Bcast(&som, 1, MPI_INT, 0, MPI_COMM_WORLD);
else
    MPI_Recv(&som, 1, MPI_INT, 0, 1000, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

d) Le processus 0 veut envoyer le message m = 100 au processus 1

```
if (rang == 0) {
    m = 100;
    MPI_Send(&m, 1, MPI_INT, 1, 1000, MPI_COMM_WORLD);
} else if (rang == 1) {
    MPI_Request req;
    MPI_Irecv(&m, 1, MPI_INT, 0, 1000, MPI_COMM_WORLD, &req);
    MPI_Wait(&req, MPI_STATUS_IGNORE);
}
```

e) Le processus 0 veut recevoir des messages en utilisant MPI_ANY_SOURCE

```
int rang, val; MPI_Status sta;

if (rang == 0) {
    MPI_Recv(&val, 1, MPI_INT, MPI_ANY_SOURCE, 1000, MPI_COMM_WORLD, &sta);
} else if (rang == 1) {
    val = rang;
    MPI_Send(&val, 1, MPI_INT, 0, 1111, MPI_COMM_WORLD);
} else if (rang == 2) {
    val = rang;
    MPI_Send(&val, 1, MPI_INT, 0, 1000, MPI_COMM_WORLD);
}
```

Exercice III : Implémentation d'un allgather

Listing de la fonction allgather et son programme principal : complétez-le.

```
1. void allgather( int in, int *out ) {
2.     int rang, P;
3.
4.     /* A COMPLETER : il faut affecter le nombre de processus dans P */
5.     /* A COMPLETER : il faut affecter la variable rang */
6.
7.     if (rang == 0) {
8.         MPI_Status sta;
9.         out[0] = in;
10.        for(int p = 1 ; p < P ; p++) {
11.            int val;
12.            /* A COMPLETER : réception dans val de la valeur envoyée par le processus de rang
13.            p */
14.            /* A COMPLETER : mise à jour du tableau out */
15.        }
16.    } else {
17.        /* A COMPLETER : envoi de in au processus 0 */
18.    }
19.    /* A COMPLETER : le processus 0 diffuse out aux autres processus */
20.}
21.int main(int argc, char *argv[])
22.{
23.    int rang, P, *out;
24.
25.    /* A COMPLETER */
26.    /* A COMPLETER : il faut affecter le nombre de processus dans P */
27.    /* A COMPLETER : il faut affecter la variable rang */
28.
29.    out = (int*)malloc(P*sizeof(int));
30.    allgather(rang, out);
31.    for(int p = 0 ; p < P ; p++)
32.        printf("P%d, out[%d] = %d\n", rang, p, out[p]);
33.    free(out);
34.
35.    /* A COMPLETER */
36.    return 0;
37.}
```

Soit la fonction : `void allgather( int in, int *out );` appelée en même temps par tous les processus MPI. Elle collecte dans le tableau `out` le contenu de toutes les variables `in` de tous les processus MPI. Tous les processus doivent récupérer les résultats. Le tableau `out` est préalablement alloué au nombre total de processus.

La fonction `allgather` est implémentée dans le listing ci-dessus, ainsi qu'un programme principal qui utilise cette fonction. Cependant le listing n'est pas complet.

Travail à faire : compléter les lignes `/* A COMPLETER */` pour que la fonction `allgather` et le programme principal soient corrects.

Exercice IV : Recouvrement

On considère une fonction **collect_trait** définie ci-dessous :

```
void collect_trait(int *tab, int N)
{
    int rank, P, k;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &P);

    if (rank == 0)
    {
        MPI_Status sta;
        int *buf2 = (int*)malloc(N*sizeof(int));

        trait(tab, N);

        for(k = 1 ; k < P ; k++)
        {
            MPI_Recv(buf2, N, MPI_INT, k, 1000, MPI_COMM_WORLD, &sta);
            trait(buf2, N);
        }

        free(buf2);
    }
    else
    {
        MPI_Send(tab, N, MPI_INT, 0, 1000, MPI_COMM_WORLD);
    }
}
```

La fonction **collect_trait** est utilisée dans les conditions suivantes :

1. cette fonction est appelée par tous les processus MPI (comme une opération collective) ;
2. chaque processus a pré-alloué et rempli son propre tableau tab de N entiers ;
3. la valeur de N est identique pour tous les processus MPI ;
4. la fonction trait(int *tab, int N) (appelée par le processus 0) effectue un traitement coûteux à partir des données du tableau tab passé en argument, sans modifier le contenu de ses données.

Question 1 : quelle est la modification à apporter à la réception pour que le processus 0 reçoive un message de n'importe quel processus ?

Question 2 : expliquez brièvement pourquoi la réception (par le processus 0) n'est pas recouverte par le traitement trait.

Question 3 : modifiez la section de code exécutée par le processus 0 pour recouvrir la réception par le traitement trait tout en recevant les messages dans n'importe quel ordre.

Indication : utilisez deux buffers de N entiers, un pour recevoir des données, l'autre pour traiter des données déjà reçues.

===== **FIN** =====